

Computer Science Project One

Rafe Milner
5/5/2025

Computer Science Project One

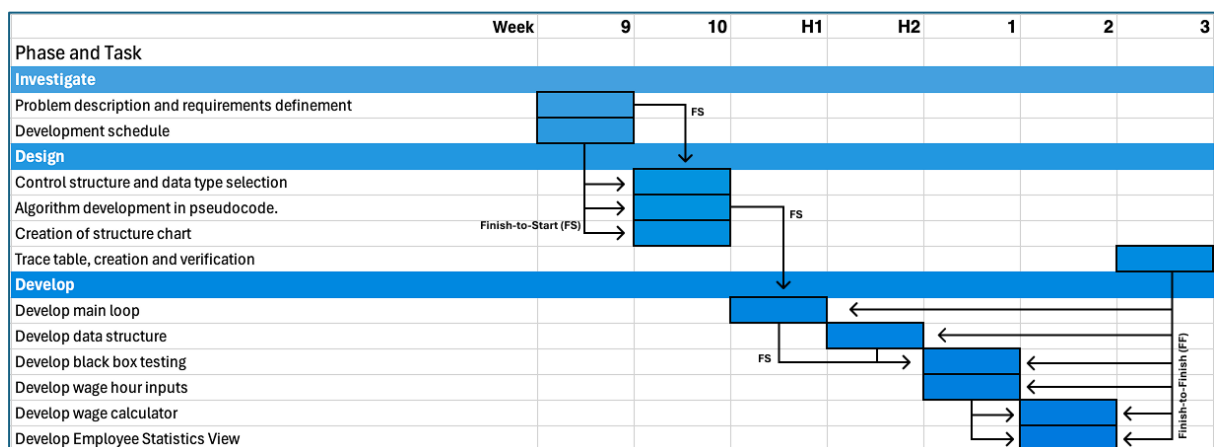
Rafe Milner

Mario Cafe, under the ownership of Mr. Mario Mushroom, currently calculates employee pay from the Monday to Sunday timeframe on Thursdays manually. This is a time consuming and complex process. Mr. Mario Mushroom would like a command line tool to be developed which allows him to calculate the wages of his employees, view the hours worked for the employee in the past week on each day and find out on which days employees worked the most.

Requirements:

-  Text-based interface (So Mario Mushroom can look like a hacker)
-  Ability to view hours worked for an employee on the past week of each day
-  Ability to enter employee name, position and hours worked
-  Ability to mark a day as a holiday
-  Pay calculations based on hours worked, overtime and holiday/weekend.

Development schedule time plan:



Program design

Control Structures:

Sequence

In this code, the sequence control structure is not used almost anywhere. This is due to the fact that this is a highly dynamic data-driven user-facing program. It is not a step-by-step linear process as it allows the user to pick which action they would like to take and then uses selection and iteration to provide an adequate user experience. A step-by-step system does not have the ability to provide a dynamic program or meet the user experience or functionality requirements for this program.

Selection

The selection control structure will be used throughout the program to select different execution paths based on relevant data, throughout the code, both if-else and switch statements have been used. If-else selection structures execute if a comparative argument is truthy and execute the else block if the argument is falsie. Switch statements match input strings to a predefined list or an exception case.

Example 1:

```
# Based on the employee's position define pay rate.  
if employeeData[index]["position"] == "Manager":  
    payrate = 30.0  
else:  
    payrate = 23.0
```

In this example, the code first grabs the employee's position. It does this by finding the employee's object by the index in the employee data list and then using dictionary notation to locate the string of their position. It then compares this to the manager role.

If the employer is a manager, the argument will return as truthy and the if statement code will be executed setting the pay rate to \$30 an hour. If the employee is not a manager, then their pay rate is set to \$23 an hour.

The selection control structure allows for the pay rate to be determined by the employee's position, selecting their pay rate based on it.

Example 2:

```
# Match the user's input position. If there isn't a position, return an error.
match position:
    case "1":
        position = "Manager"
    case "2":
        position = "Barista"
    case "3":
        position = "Cleaner"
    case _:
        position = ""
        errormessage = "Please select one of the options; [1]-Manager, [2]-Barista, [3]-Cleaner"
        continue
```

In this example, the selection control structure is given a string from the user's input. If the user has inputted a number from 1 to 3, it sets the equivalent position to the position variable. By using the underscore case, which is the catchall case, if the user enters a non-supported number, an error message will be returned to them.

The selection control structure allows for user input validation. Instead of requiring the user to type in the full position, they simply must enter a number which is efficiently mapped to the correct position. This control structure also comes in with built-in exception handling, using the catchall case which allows for an error message to be responded to the user.

Example 3:

```
# Have the first day Feature a message referencing the Employees's name.
if day == "Monday":
    inputString = f"Enter the hours worked by {employeeData[index]['name']} for {day}: "
else:
    inputString = f"Enter the hours worked for {day}: "
```

In this example, for user experience reasons, on the first day, which is Monday, the user is presented by the employee's name and the day. But on subsequent requests, it is assumed that the user knows who the employee is. Therefore, an if statement selects which string should be used based on whether the day is Monday or any other day.

Iteration

The iteration control structure will be used throughout the program to allow for data structures to be traversed and to iterate and change lists. It allows for a sequence of code to be executed a set number of times based on the size of a data object which is very useful. For example, an array, a list of all the days in the week can be iterated through and code to calculate the pay for each day of the week can be run on each iteration.

Example 1:

```
# Using iteration control structure, check whether the name provided is in the list of employees.
for index, employee in enumerate(employeeData):
    # If the name is in the list of employees, return false and provide the index to that data.
    # Strings modified to remove spaces and lowercase all uppercase characters.
    if "".join(name.split()).lower() == "".join(employee["name"].split()).lower():
        return True, index
    else:
        continue
```

In this example of the iteration control structure, the employee data list which holds employee objects is iterated through. For each of the employees, their name is checked against the name that the user has inputted to ensure that there are no data collisions. By iterating through the employee data list, which contains employee objects, the list can be as large or as small as required and each name will still be efficiently and adequately checked for collisions.

Example 2:

```
days = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]

for dayNumber, day in enumerate(days):
```

In this example, a variable of days has been defined with an array of all the days in a week. An iteration control structure of a for loop is then used to run code for every day in a week. This provides a more efficient way for running code for each day in the week compared to a sequential control structure, which would have to duplicate code to reach this effect.

Data Types:

List:

The list data type, also known as array, is a key data type as it allows for a set of data to be assigned to one variable and then for that set to be iterated through using a for loop and other iteration control structures.

`someVariable = [ "hello", 1, 6.7, True, ["arrays!!", "in arrays"] ]`

lists are defined by a variable name equalling brackets with elements inside those brackets. The elements can be strings, integers, floats, booleans or other arrays.

`someVariable[2] → 6.7`

They can be accessed by the variable name with square bracket notation for the index of the item that should be retrieved, with indexing starting at zero.

Example – days list:

```
days = [  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday",  
    "Saturday",  
    "Sunday",  
]
```

This list provides two things to the program. Firstly, it provides an array that can be iterated over for each of the seven days, conveniently allowing code to be run on each

day. It also contains strings of the names of each day which can be used to communicate with the user.

String:

The string data type stores text, this text can be printed, modified or converted into other data types. Strings are the primary data type, especially in user-driven apps as input function return strings.

someVariable = "This is a string"

Strings are defined by variable name equalling text contained in quotation marks.

print(someVariable) → This is a string

They can be accessed by calling the variable.

Example – Employee name input:

```
# Try except expression for name input to allow for clean exiting, back to the main screen
try:
    nameInput = input(
        "What is the name of the employee you would like to add: "
    )
except KeyboardInterrupt:
    clear()
    break

# Check if the user would like to return to the homepage.

if nameInput.lower() == "b":
    clear()
    break

# Check if the employee already exists.
employeeExists, indexOfEmployee = checkEmployee(nameInput)

if employeeExists:
    errorMessage = "That name is already used, Enter a different name."
else:
    name = nameInput
    continue
```

This string is what the user would like to call the employee they are creating. Any combination of characters should work, but to avoid data collisions it is imperative that names that have been used before are not used again.

Input validation:

```
# Function to check whether an employee exists in the data structure.
def checkEmployee(name):
    # IntelliSense information
    """
    Check whether an employee exists in the data structure

    Returns:
    boolean: True = Employee does not exist / False = employee does exist
    int: The index of the employee
    """

    # Using iteration control structure, check whether the name provided is in the list of employees.
    for index, employee in enumerate(employeeData):
        # If the name is in the list of employees, return false and provide the index to that data.
        # Strings modified to remove spaces and lowercase all uppercase characters.
        if "".join(name.split()).lower() == "".join(employee["name"].split()).lower():
            return True, index
        else:
            continue

    # If the name is not in the data structure, return true and do not provide an index.
    return False, None
```

To validate the input, a function is run which takes the string input as an argument. It iterates through the employee data structure and checks each employee object for the name string. It then converts both the current iteration object string and the input string to lowercase and removes all spaces to compare whether they are the same. If they are, then it informs the user via a selection control structure to pick a different name that does not cause a data collision.

Float

The float data type stores numbers with floating point position, which means that float numbers support decimal places, allowing for fractions, percentages and minute data changes.

someVariable = 6.7

Strings are defined by variable name equalling a number with a decimal point.

print(someVariable) → 6.7

They can be accessed by calling the variable.

Example – Hours worked in a day input:

```
# Error handling for float numbers.
try:
    hoursworked = float(input(inputString))
except ValueError:
    internalErrorMessage = (
        "Please enter a number between 0 and 24 +"
    )
    continue

# Check whether the input is in range.
if hoursworked < 0.0 or hoursworked > 24.0:
    internalErrorMessage = "Please enter a number between 0 and 24"
    continue
else:
    employeeData[index]["hourspersday"][dayNumber] = hoursworked
```

The float input indicates how many hours an employee has worked in a certain day. This input is a float, not an integer because employees could sometimes work 1 hour and 30 minutes which would be 1.5 in float terms but could not be expressed using an integer.

Input validation:

```
# Check whether the input is in range.
if hoursworked < 0.0 or hoursworked > 24.0:
    internalErrorMessage = "Please enter a number between 0 and 24"
    continue
else:
    employeeData[index]["hourspersday"][dayNumber] = hoursworked
```

As the user is entering hours worked in a day, the lowest number of hours that an employee can work is zero and the highest in a day is 24 hours. To check that, a selection control structure is used to check whether hours worked is under zero or over 24. and if it is, it presents an error message to the user and asks them to re-enter the number. If hours worked is within the range, it is added to the employee's objects and appended to the Employee data structure.

Exception handling:

```
# Error handling for float numbers.
try:
    hoursworked = float(input(inputString))
except ValueError:
    internalErrorMessage = (
        "Please enter a number between 0 and 24 +"
    )
    continue
```

Not all inputs made by a user into the input can be converted into Float numbers. Therefore, a try accept block is deployed to handle a user entering an input that cannot be converted into a float. If this occurs, the except value error block will catch it and return an error to the user.

Dictionary

The dictionary data type is much like the list data type as it stores sets of data. But unlike lists, which are accessed by using a number to index into them, dictionaries provide keys and values where keys are string naming the data and values are the data. For example, you may have a key name and then the name of the person of the dictionary object.

someVariable = { "name": "Rafe Milner" }

Dictionaries are defined by a variable name equally, squiggly brackets with content within the brackets following a key in string form, then a colon, then any data type ranging from list to float to string.

print(someVariable["name"]) → Rafe Milner

They can be accessed by calling the variable with the string of the key in square brackets.

Example – Creating employee objects(dictionaries):

```
# Create dictionary based on the given information then calculate its index in the employee data array.
employeeObject = dict(
    name=name,
    position=position,
    hoursperday=[0, 0, 0, 0, 0, 0, 0],
    holidays=[False, False, False, False, False, False, False],
    payperday=[0, 0, 0, 0, 0, 0, 0],
    totalpay=0,
)
employeeData.append(employeeObject)
index = len(employeeData) - 1
```

For each employee there is a dictionary that defines their name, position, hours per day, holidays, pay per day and total pay. This is created using the dict function which creates a dictionary which is then appended to the employee data list structure. By using a dictionary as the object for each employee, the attributes related to that employee are bundled together, meaning that there are no race conditions or out of sync bugs. And it also means that data is easily accessed. For example, if you have the name of the employee, the employee's other attributes are easy to find.

Algorithms

Main Loop Function algorithm

```
1  FUNCTION Main
2      errormessage = ""
3
4      WHILE TRUE:
5          CLEAROUTPUT()
6
7          functionnumber = int(
8              input( /START multiline templating string/
9
10             Welcome to Mushroom Cafe Payroll Management:
11
12             Today is {TODAYSDATE()})
13
14             What function would you like to perform?
15             (1) - Enter employee hours.
16             (2) - View employee hours and pay.
17             (3) - Exit
18
19             {errormessage}
20
21             Enter the function number and press enter to continue: /END multiline templating string/)
22         )
23     CATCH ValueError
24         errormessage = "Please enter the number of the function you would like to perform 1, 2 or 3"
25         CONTINUE
26     END CATCH
27
28     CATCH KeyboardInterrupt
29         CLEAROUTPUT()
30         EXIT
31     END CATCH
32
33     IF functionnumber == 1
34         function1()
35     ELIF functionnumber == 2
36         function2()
37     ELIF functionnumber == 3
38         CLEAROUTPUT()
39         EXIT
40     ELSE
41         errormessage = "Please enter the number of the function you would like to perform 1, 2 or 3"
42         continue
43     END IF
```

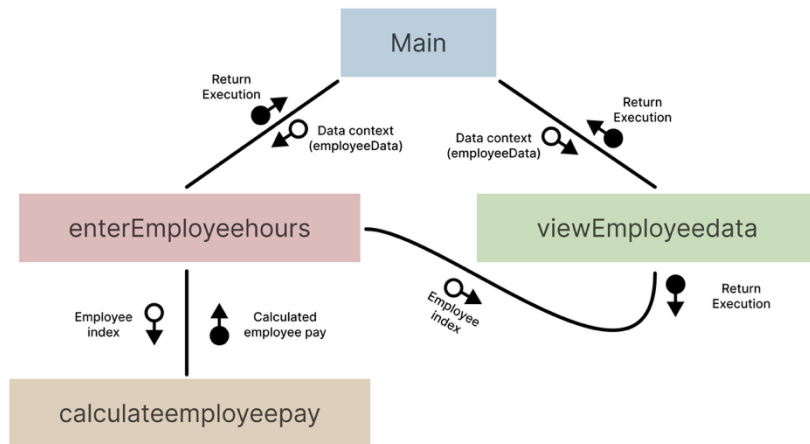
User Input for Hours per day algorithm

```
1  FOR dayNumber = 0 TO days.LENGTH()
2
3  day = days[dayNumber]
4  internalErrorMessage = ""
5
6  WHILE TRUE
7      IF day = "Monday"
8          |   inputString = f"Enter the hours worked by {employeeData[index]['name']} for {day}:  "
9      ELSE
10         |   inputString = f"Enter the hours worked for {day}:  "
11     END IF
12
13     IF internalErrorMessage:
14         |   PRINT(internalErrorMessage)
15     END IF
16
17
18     hoursworked = float(input(inputString))
19     CATCH ValueError
20         |   internalErrorMessage = (
21         |       "Please enter a number between 0 and 24"
22         |   )
23         |   CONTINUE-TO-NEXTLOOP-ITERATION
24     END CATCH
25
26     IF hoursworked < 0.0 or hoursworked > 24.0
27         |   internalErrorMessage = "Please enter a number between 0 and 24"
28         |   CONTINUE-TO-NEXTLOOP-ITERATION
29     ELSE
30         |   employeeData[index]["hoursperday"][dayNumber] = hoursworked
31     END IF
32
33     internalinternalErrorMessage = ""
34
35     WHILE TRUE
36         IF internalinternalErrorMessage
37             |   PRINT(internalinternalErrorMessage)
38         END IF
39
40         holidayInput = input(f"Is {day}, a holiday? [Y]es/[N]o:  ")
41
42         IF holidayInput.lower() = "yes" or holidayInput.lower() = "y"
43             |   employeeData[index]["holidays"][dayNumber] = TRUE
44             |   BREAK
45         ELIF holidayInput.lower() = "no" or holidayInput.lower() = "n":
46             |   BREAK
47         ELSE
48             |   internalinternalErrorMessage = "Please enter yes or no"
49             |   CONTINUE-TO-NEXTLOOP-ITERATION
50         END WHILE
51
52     END WHILE
53 END FOR
```

Payment Calculation algorithm

```
1 FUNCTION calculateEmployeePay(index)
2   days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
3
4   FOR dayNumber = 0 TO days.LENGTH()
5
6       day = days[dayNumber]
7
8       # Define pay variable
9       pay = 0.0
10
11       # Based on the employee's position define pay rate.
12       IF employeeData[index]["position"] = "Manager"
13           payrate = 30.0
14       ELSE
15           payrate = 23.0
16       END IF
17
18       # Define overtime rates.
19       overtime1 = 25.0
20       overtime2 = 45.0
21
22       # Change overtime and pay rates for weekends.
23       # If it is the weekend or a holiday, Change overtime to 50% and change pay rate.
24       IF day = "Sunday" OR employeeData[index]["holidays"][dayNumber] OR day = "Saturday"
25
26           # If it is a holiday or Sunday, set the pay rate to 4.
27           # If it's not, which would be Saturday, set it to 3.
28           IF day = "Sunday" OR employeeData[index]["holidays"][dayNumber]
29               payrate = payrate + 4
30           ELSE
31               payrate = payrate + 3
32           END IF
33
34           # Set overtime to weekend holiday values.
35           overtime1 = 50.0
36           overtime2 = 50.0
37       END IF
38
39       # If the employee has worked less than the overtime threshold on that day;
40       IF employeeData[index]["hoursperday"][dayNumber] ≤ 9.0
41
42           # Then their hours worked is simply multiplied by the pay rate,
43           # which has already been adjusted to include the weekend and holiday benefits.
44           pay = employeeData[index]["hoursperday"][dayNumber] * payrate
45
46           # The data is then saved into the array value for that day.
47           employeeData[index]["payperday"][dayNumber] = pay
48
49           # And the total pay for the weekly period is then calculated by summing that array.
50           totalpay = SUM(employeeData[index]["payperday"])
51           employeeData[index]["totalpay"] = totalpay
52
53       ELSE
54
55           # If the employee has worked more than 9 hours,
56           # then we subtract 9 hours from the hours worked and calculate
57           # those 9 hours at the normal pay rate for them.
58           hoursworked = employeeData[index]["hoursperday"][dayNumber]
59           hoursworked = hoursworked - 9
60           pay = 9 * payrate
61
62           # If the employee has worked more than three hours, putting them in the higher band of overtime,
63           IF hoursworked > 3
64
65               # We first remove the three hours from the hours worked and then calculate those three hours
66               # then increase it by the overtime amount.
67               hoursworked = hoursworked - 3
68               pay = pay + (3 * payrate) * (1 + overtime1 / 100)
69
70               # Finally, the remaining hours are multiplied by the pay rate and then multiplied by
71               # the higher overtime rate.
72               pay = pay + (hoursworked * payrate) * (1 + overtime2 / 100)
73
74           ELSE
75
76               # If the employee has not worked enough hours to go into the higher band of overtime,
77               # then the amount of hours they worked is times by their pay rate and increased by the lower
78               pay = pay + (hoursworked * payrate) * (1 + overtime1 / 100)
79
80           END IF
81
82           # The data is then saved into the array value for that day.
83           employeeData[index]["payperday"][dayNumber] = pay
84
85           # And the total pay for the weekly period is then calculated by summing that array.
86           totalpay = SUM(employeeData[index]["payperday"])
87           employeeData[index]["totalpay"] = totalpay
88
89       END IF
90
91   END FOR
92
93 END FUNCTION
```

Structure Chart



Trace Tables

Rosa Lina - Manager - [4,4,4,4,14,0,12]					
Day	Total Pay	Payrate	Overtime 1	Overtime 2	Hours per day
Monday	0				
Monday	0	30	25	45	
Monday	0	30	25	45	4
Monday	120	30	25	45	4
Tuesday	120				
Tuesday	120	30	25	45	
Tuesday	120	30	25	45	4
Tuesday	240	30	25	45	4
Wednesday	240				
Wednesday	240	30	25	45	
Wednesday	240	30	25	45	4
Wednesday	360	30	25	45	4
Thursday	360				
Thursday	360	30	25	45	
Thursday	360	30	25	45	4
Thursday	480	30	25	45	4
Friday	480				
Friday	480	30	25	45	14
Friday	750	30	25	45	5
Friday	862.5	30	25	45	2
Friday	949.5	30	25	45	2
Saturday	949.5				
Saturday	949.5	33	50	50	0
Sunday	949.5				
Sunday	949.5	34	50	50	12
Sunday	1255.5	34	50	50	3
Sunday	1408.5	34	50	50	3

Cooper Trouper - Barista - [9, 9, 9, 0, 9, 6, 10]					
Day	Total Pay	Payrate	Overtime 1	Overtime 2	Hours per day
Monday	0				
Monday	0	23	25	45	
Monday	0	23	25	45	9
Monday	207	23	25	45	9
Tuesday	207				
Tuesday	207	23	25	45	
Tuesday	207	23	25	45	9
Tuesday	414	23	25	45	9
Wednesday	414				
Wednesday	414	23	25	45	
Wednesday	414	23	25	45	9
Wednesday	621	23	25	45	9
Thursday	621				
Thursday	621	23	25	45	
Thursday	621	23	25	45	0
Friday	621				
Friday	621	23	25	45	9
Friday	828	23	25	45	9
Saturday	828				
Saturday	828	26	50	50	6
Saturday	984	26	50	50	6
Sunday	984				
Sunday	984	27	50	50	10
Sunday	1227	27	50	50	1
Sunday	1267.5	27	50	50	1

Testing

Employee Hour Input:

Requirements for input:

Any positive number, including decimals, between 0 and 24 (inclusive).

Test Table:

Test ID	Input	Type	Expected outcome	Reason
T1	"Hello World"	Invalid input type	Error message (Not accepted input type)	String is not an accepted type.
T2	"Smart Cookie"	Invalid input type	Error message (Not accepted input type)	String is not an accepted type.
T3	"67"	Invalid in range	Error message (Not in Range)	Number is out of range.
T4	0	Boundary	Accepted	
T5	24	Boundary	Accepted	
T6	31	Invalid in range	Error message (Not in Range)	Number is out of range.
T7	10	Valid in range	Accepted	
T8	4	Valid in range	Accepted	
T9	9	Valid in range	Accepted	

Function Select Input:

Requirements for input:

A number of one, two or three.

Test Table:

Test ID	Input	Type	Expected outcome	Reason
T1	"Hello World"	Invalid input type	Error message (Not accepted input type)	String is not an accepted type.

T2	"Smart Cookie"	Invalid input type	Error message (Not accepted input type)	String is not an accepted type.
T3	"67"	Invalid	Error message (non-accepted input)	Input is not 1, 2 or 3
T4	0	Invalid	Error message (non-accepted input)	Input is not 1, 2 or 3
T5	1	Valid	Accepted	
T6	2	Valid	Accepted	
T7	3	Valid	Accepted	
T8	4	Invalid	Error message (non-accepted input)	Input is not 1, 2 or 3
T9	5	Invalid	Error message (non-accepted input)	Input is not 1, 2 or 3